



פרויקט מסד נתונים

הנושא: קונדיטוריה

שם המורה: א.רוזן

אפרת מני

תוכן עיניינים

מבוא

1. תיאור הארגון-

2. מטרות הפרויקט-

תיאור הנתונים

1. דיאגרמת קשרים-

2. הסבר קשרים-

3. דיאגרמת ERD-

4. אילוצי נתונים-

הפקת מידע וביצוע תהליכים –

1. שאילתות-

2. VIEW-

3. פונקציות סקלריות-

4. פונקציות המחזירות טבלה-

5. פרוצדורות-

6. טריגרים-

מבוא

תיאור הארגון -

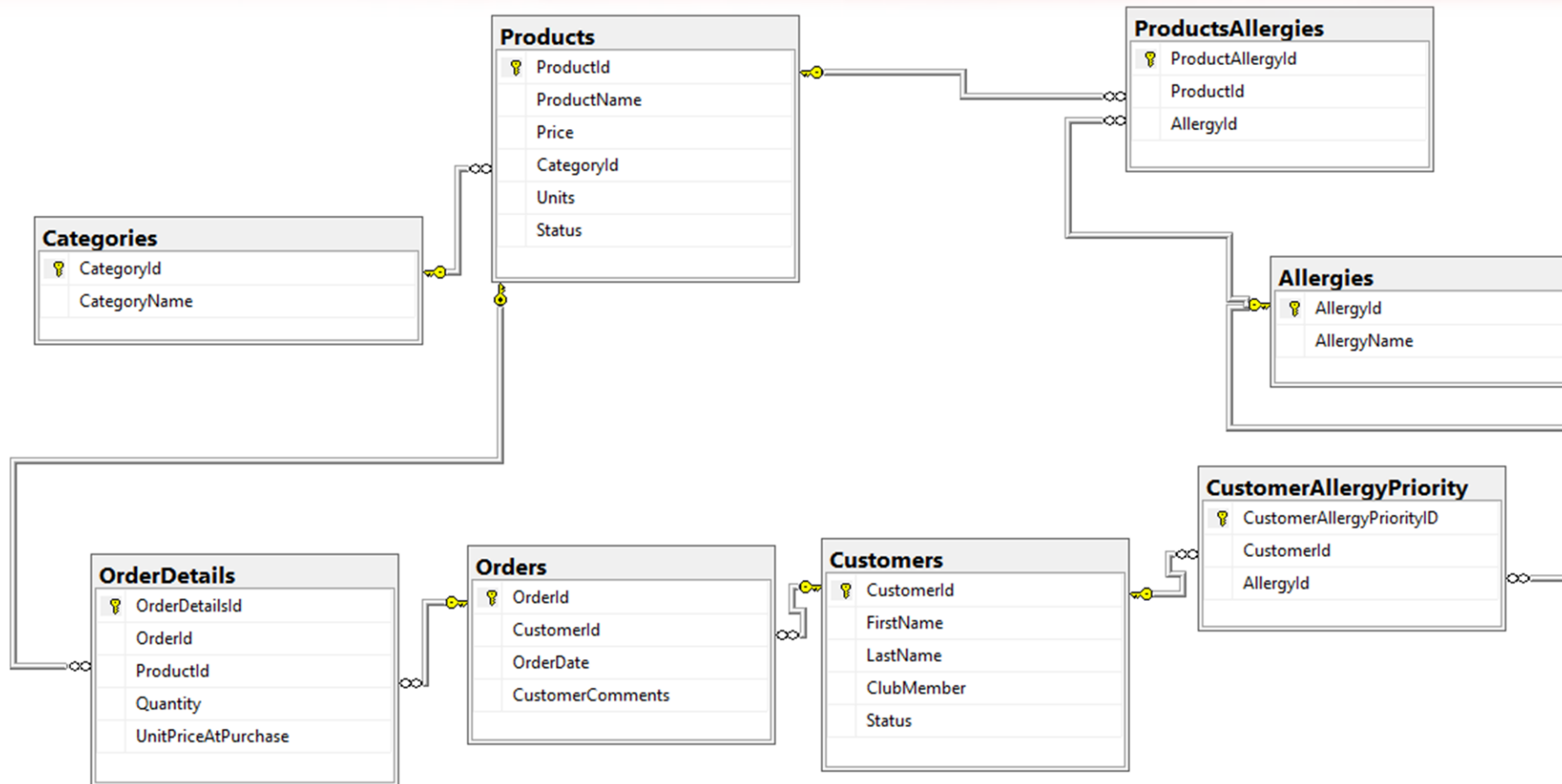
הקונדיטוריה עוסקת במכירת מוצרים מוכנים ללקוחות פרטיים. בין המוצרים הנמכרים ניתן למצוא עוגות, עוגיות, קינוחים אישיים, מאפים מסוגים שונים ועוד. כל מוצר שייך לקטגוריה מסוימת, מתומחר בהתאם, ומוצגת כמות זמינה ממנו במלאי.

הקונדיטוריה מתאפיינת ברגישות מיוחדת לנושא האלרגיות, ולכן כל מוצר מקושר לאלרגנים שהוא עשוי להכיל, כגון חלב, גלוטן, אגוזים, בוטנים וכדומה. מידע זה זמין ללקוחות כדי לאפשר רכישה בטוחה ומותאמת אישית. לקוחות הקונדיטוריה יכולים לבצע הזמנות, הכוללות פריטים מסוגים שונים ובכמויות משתנות. חלק מהלקוחות הם חברי מועדון, הזכאים לשירותים או תנאים מיוחדים. במהלך הרכישה ניתן להוסיף הערות אישיות לביצוע ההזמנה. מערכת המידע של הארגון מרכזת נתונים על לקוחות, הזמנות, מוצרים, קטגוריות ואלרגנים. בעזרת המערכת ניתן לעקוב אחר היסטוריית רכישות, ניתוח העדפות, ניהול מלאי, והתאמה לצרכים בריאותיים של הלקוחות.

מטרות הפרויקט:

1. ניהול מוצרים וקטגוריות
2. ניהול לקוחות והזמנות
3. מערכת אלרגנים
4. ניתוח מועדון הלקוחות
5. דוחות ניתוח מכירות

דיאגרמת קשרים (DSD)



הסבר קשרים

מפתח ראשי

לכל טבלה ישנו שדה שהוא המזהה הייחודי לאותה טבלה.

CategoryId -	בטבלת Category
AllergyId -	בטבלת Allergy
ProductId -	בטבלת Product
OrderId -	בטבלת Order
CustomerId -	בטבלת Customer
ProductsAllergiesId -	בטבלת ProductsAllergies
OrderDetails -	בטבלת OrderDetailsId

מפתח זר

רבים ליחיד –

1. לכל מוצר בקונדיטוריה יש קטגוריה, ומכיוון שקטגוריה אחת יכולה להיות אצל מוצרים רבים זהו קשר רבים ליחיד.
2. כל לקוח הרשום במערכת יכול לבצע יותר מהזמנה אחת, לכן זהו קשר רבים ליחיד.

רבים לרבים –

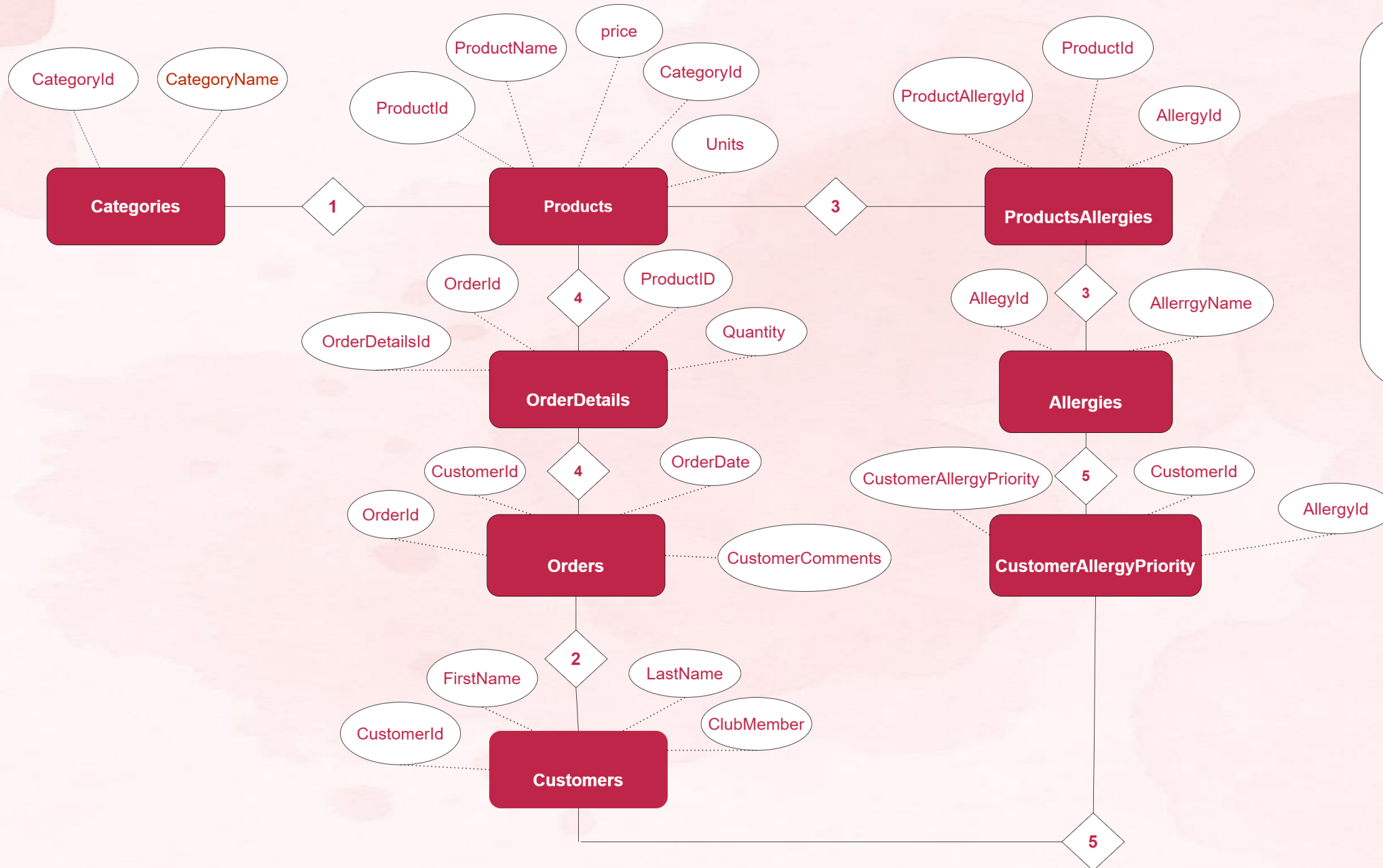
3. ישנם מוצרים בעלי אלרגנים. לכל מוצר יכול להיות יותר מאלרגן אחד, לכן זהו קשר רבים לרבים.
4. לכל הזמנה יש את המוצרים שהוזמנו בה. בכל הזמנה יכול להיות יותר ממוצר אחד, לכן זהו קשר רבים לרבים.
5. כל לקוח יש את האלרגנים שהוא לא מעוניין בהם (אלרגי). ללקוח יכול להיות יותר מאלרגן אחד, ואלגן אחד יכול להיות אצל יותר מלקוח אחד.

דיאגרמת ERD -

מקרא מפה

ישות חזקה

תכונה
מפתח זר —



אילוצי נתונים

1. `CONSTRAINT UQ_ProductAllergy UNIQUE (ProductId, AllergyId)`
2. `Price FLOAT CHECK (Price > 0) NOT NULL`
3. `OrderDate DATETIME DEFAULT CURRENT_TIMESTAMP`
4. `Units INT CHECK (Units > 0) NOT NULL`
5. `CONSTRAINT UQ_ProductAllergy UNIQUE (ProductId, AllergyId)`

1. למוצר אחד אין פעמיים את אותה אלרגן.
2. מחיר המוצר חייב להיות חיובי.
3. בעת הזמנה התאריך והשעה הנוכחיים יכנסו לשדה OrderDate.
4. הכמות במלאי חייבת להיות חיובית.
5. לכל מוצר אלרגן יכול להופיע פעם אחת בלבד.

שאלות -

תיאור: שליפת הלקוחות שבצעו הזמנה ומספר ההזמנות. סדר הצגתם לפי כמות ההזמנות של כל אחד. (מענה למטרה 2)

```
-- שליפת הלקוחות שבצעו הזמנה ומספר ההזמנות שבצעו וסדר הצגת לפי כמות ההזמנות שהזמינו
select c.FirstName + ' ' + c.LastName as FullName, COUNT(o.OrderId) as NumOfOrders
from Customers c join Orders o on c.CustomerId = o.CustomerId
group by c.FirstName, c.LastName
order by COUNT(o.OrderId) desc
```

דוגמה לפלט:

	FullName	NumOfOrders
1	אפרת מני	2
2	טובה גולד	1
3	נועה גולדשטיין	1

שאלות -

תיאור: מיון לפי מחיר + הצגת אלרגנים. (מענה למטרה 1 ו- 3)

מיון לפי מחיר + הצגת אלרגנים--

```
select p.ProductId, p.ProductName, p.Price, p.CategoryId, al.AllergyName, p.Units
from Products p left join ProductsAllergies pal on p.ProductId = pal.ProductId
left join Allergies al on pal.AllergyId = al.AllergyId
where p.Status = 1
order by p.Price asc
```

דוגמה לפלט:

	ProductId	ProductName	Price	CategoryId	AllergyName	Units
1	24	טראפל שוקולד	5	2	NULL	35
2	1	מוס שוקולד בלגי אישי	12.9	2	גלוטן	50
3	18	עוגת גבינה	32	1	חלב	6
4	20	קראוסון חמאה	32	1	חלב	6
5	20	קראוסון חמאה	32	1	גלוטן	6
6	6	עוגת קפה	33	1	NULL	5
7	15	עוגת שוקולד	35	1	חלב	12
8	23	פאי פיצוחים	60	2	גלוטן	30
9	23	פאי פיצוחים	60	2	אגוזים	30
10	23	פאי פיצוחים	60	2	בוטנים	30

שאלות -

תיאור: שליפת כמות ההזמנות של לקוח מסוים. (מענה למטרה 2)

--שליפת כמות ההזמנות של לקוח מסוים

```
declare @CustomerId int
```

```
set @CustomerId = 2
```

```
Select c.FirstName + ' ' + c.LastName as FullName , count(o.OrderId)
```

```
from Customers c join Orders o on c.CustomerId = o.CustomerId
```

```
where c.CustomerId = @CustomerId
```

```
group by c.FirstName, c.LastName
```

דוגמה לפלט:

	FullName	(No column name)
1	טובה גולד	1

שאלות -

תיאור: שליפת חברי מועדון פעילים.
(מענה למטרה 4)

```
-- שליפת כל חברי המועדון הפעילים
Select *
from Customers
where ClubMember = 1 AND Status = 1
```

דוגמה לפלט:

	CustomerId	FirstName	LastName	ClubMember	Status
1	1	אפרת	מני	1	1
2	4	הדס	קליין	1	1

תיאור: שלוף את הלקוח שבצע את ההזמנה אחרונה. (מענה למטרה 2)

```
-- מצא את הלקוח שביצע את ההזמנה האחרונה
SELECT TOP 1 c.FirstName + ' ' + c.LastName as 'הלקוח שבצע את הזמנה האחרונה', o.OrderDate
FROM Orders o join Customers c on o.CustomerId = c.CustomerId
ORDER BY OrderId DESC
```

דוגמה לפלט:

	הלקוח שבצע את הזמנה האחרונה	OrderDate
1	נועה גולדשטיין	2025-05-22 11:50:11.540

שאלות -

תיאור: שליפה כל ההזמנות. (מענה למטרה 2)

שליפה של טבלת מוצרים מלאה--

```
select p.ProductId, p.ProductName, p.Price, c.CategoryName, p.Units
from Products p join Categories c on p.CategoryId = c.CategoryId
```

שליפה של טבלת הזמנות מלאה--

```
select c.FirstName + ' ' + c.LastName as FullName, o.OrderDate, o.CustomerComments
from Orders o inner join Customers c on o.CustomerId = c.CustomerId
order by o.OrderDate asc
```

דוגמה לפלט:

	FullName	OrderDate	CustomerComments
1	אפרת מני	2025-05-18 15:53:25.543	תודה רבה!
2	טובה גולד	2025-05-18 15:53:25.543	NULL
3	נועה גולדשטיין	2025-05-22 11:50:11.537	אני אלרגית לבוטנים, אנא ודאו שאין עקבות של אגוזים
4	נועה גולדשטיין	2025-05-22 11:50:11.540	NULL

שאלות -

תיאור: שליפת כל המוצרים (פעילים) שעולים מחיר מסוים. (מענה למטרה 1)

```
-- הצגת כל המוצרים שעולים מחיר מסוים
declare @price FLOAT
set @price = 39.9

select *
from Products p
where p.Price = @price AND P.Status = 1
```

דוגמה לפלט:

	ProductId	ProductName	Price	CategoryId	Units	Status
1	1	מוס שוקולד בלגי אישי	39.9	2	50	1
2	19	טריקולד	39.9	4	32	1

שאלות -

תיאור: שליפת כל הלקוחות שקנו מעל 200. (מענה למטרה 5)

```
--שליפת כל הלקוחות עם סך רכישות גבוה מ-200
select c.FirstName, c.LastName, SUM(od.UnitPriceAtPurchase * od.Quantity)
from Customers c join Orders o on o.CustomerId = c.CustomerId
join OrderDetails od on od.OrderId = o.OrderId
group by c.FirstName, c.LastName
having SUM(od.UnitPriceAtPurchase * od.Quantity) >200
```

דוגמה לפלט:

	FirstName	LastName	(No column name)
1	נועה	גולדשטיין	218.4

- VIEWS

תיאור: הוויו שולף קוד מוצר, שם מוצר, קוד קטגוריה, מחיר, כמות היחידות שנותרו, קוד אלרגן.

```
-- שולפת קוד מוצר, שם מוצר, קוד קטגוריה, מחיר, כמות היחידות שנותרו, קוד אלרגן--  
CREATE VIEW ProductProductsAllergiesAllergies  
AS  
select p.ProductId, p.ProductName, p.CategoryId, p.Price, p.Units, a.AllergyId  
from Products p  
join ProductsAllergies pal on p.ProductId = pal.ProductId  
join Allergies a on a.AllergyId = pal.AllergyId  
where p.Status = 1
```

דוגמה לפלט:

```
select * from ProductProductsAllergiesAllergies
```

	ProductId	ProductName	CategoryId	Price	Units	AllergyId
1	3	עוגת גבינה פרווים	3	25	30	1
2	6	קראוסון	1	8	30	1
3	6	קראוסון	1	8	30	2
4	7	עוגת גלידה	2	50	20	2

- VIEWS

תיאור: שליפת קוד הזמנה, קוד מזמין, תאריך הזמנה, כמות, קוד קטגוריה, מחיר בעת רכישה.

```
CREATE VIEW OrderOrderDetailsProducts
```

```
AS
```

```
select o.OrderId, o.CustomerId, o.OrderDate, od.ProductId, od.Quantity, p.CategoryId, od.UnitPriceAtPurchase  
from Orders o  
join OrderDetails od on o.OrderId = od.OrderId
```

```
select * from OrderOrderDetailsProducts
```

פלט:

	OrderId	CustomerId	OrderDate	ProductId	Quantity	CategoryId	UnitPriceAtPurchase
1	1	1	2025-05-04 02:38:17.280	6	1	1	8
2	1	1	2025-05-04 02:38:17.280	3	1	3	25
3	2	2	2025-05-04 02:38:20.777	3	1	3	25

פונקציות סקרליות -

```
CREATE FUNCTION dbo.IsClubMembersBuyMore()  
RETURNS BIT
```

```
AS
```

```
BEGIN
```

```
    declare @member float
```

```
    declare @notMember float
```

```
-- ממוצע קניה לחבר מועדון
```

```
    set @member = ISNULL((select SUM(od.UnitPriceAtPurchase * od.Quantity) / COUNT(DISTINCT c.CustomerId)  
                           from Customers c join Orders o on o.CustomerId = c.CustomerId  
                           join OrderDetails od on od.OrderId = o.OrderId  
                           where c.ClubMember = 1),0)
```

```
-- ממוצע קניה ללקוח שאינו חבר מועדון
```

```
    set @notMember = ISNULL((select SUM(od.UnitPriceAtPurchase * od.Quantity) / COUNT(DISTINCT c.CustomerId)  
                              from Customers c join Orders o on o.CustomerId = c.CustomerId  
                              join OrderDetails od on od.OrderId = o.OrderId  
                              where c.ClubMember = 0),0)
```

```
    IF(@member > @notMember)
```

```
        RETURN 1
```

```
    RETURN 0
```

```
END
```

```
select dbo.IsClubMembersBuyMore()
```

תיאור: הפונקציה

בודקת האם

חברי מועדון

מבצעים קניות

בממוצע בסכום

גבוה יותר מאשר

לקוחות שאינם

חברי מועדון.

1 – כן

0 – לא

(מענה למטרה 4)

דוגמה פלט:

	(No column name)
1	1

פונקציות סקלריות -

תיאור: הפונקציה מקבלת שם של מוצר ומחזירה את הקוד שלו. (מענה למטרה 1)

```
-- פונקציה המקבלת שם של מוצר ומחזירה את הקוד מוצר שלו--
CREATE FUNCTION dbo.GetProductId(@ProductName NVARCHAR(25))
RETURNS INT
AS
BEGIN
    declare @productId INT;

    IF @ProductName IS NOT NULL AND EXISTS(select 1 from Products where ProductName = @ProductName) -- מחזיר האם יש שורה שעונה על התנאי--
    BEGIN
        select TOP 1 @productId= p.ProductId
        from Products p
        where p.ProductName = @ProductName
        RETURN @productId
    END
    RETURN NULL;
END
```

```
-- קריאה לפונקציה--
select dbo.GetProductId('טריקולד') as 'ProductId'
```

דוגמה לפלט:

	ProductId
1	19

פונקציות סקלריות -

תיאור: הפונקציה מחזירה את היום הרווחי ביותר (היום עם סך הרווח הגבוה ביותר) בחודש ושנה ספציפיים. (מענה למטרה 5)

```
CREATE FUNCTION dbo.GetTheMostProfitableDay(@year INT, @month INT)
RETURNS DATE
AS
BEGIN
    DECLARE @result DATE

    SET @Result = (
        SELECT TOP 1 CAST(o.OrderDate AS DATE)
        from Orders o join OrderDetails od on o.OrderId = od.OrderId
        where DATEPART(year, o.OrderDate) = @year AND DATEPART(month, o.OrderDate) = @month -- מוציא מהתאריך רק את החודש והשנה
        group by CAST(o.OrderDate AS DATE) -- מאחד את כל השורות של אותו יום ביחד
        order by SUM(od.UnitPriceAtPurchase * od.Quantity) DESC
    )

    RETURN @result
END
```

```
-- קריאה לפונקציה--
select dbo.GetTheMostProfitableDay(2025, 5) as 'היום הכי רווחי'
```

דוגמה לפלט:

	היום הכי רווחי
1	2025-05-22

פונקציות סקלריות -

-- פונקציה המקבלת קוד הזמנה ומחזירה את המחיר הסופי של הקניה--

```
CREATE FUNCTION dbo.FinalOrderPrice(@OrderId INT)
RETURNS Money
AS
BEGIN
    Declare @FinalPrice Money
    Declare @ClubMember BIT
    IF @OrderId IS NOT NULL AND EXISTS (select 1 from Orders where OrderId = @OrderId)
    BEGIN
        select @FinalPrice = SUM(od.UnitPriceAtPurchase * od.Quantity)
        from OrderDetails od
        where od.OrderId = @OrderId
        IF Exists(select 1 from Orders o
                join Customers c on o.CustomerId = c.CustomerId
                where c.ClubMember = 1 AND o.OrderId = @OrderId)
        BEGIN
            RETURN @FinalPrice * 0.95
        END
        RETURN @FinalPrice
    END
    RETURN NULL
END
```

```
select dbo.FinalOrderPrice(4) AS FinalPrice;
```

תיאור:

הפונקציה מקבלת קוד
הזמנה ומחזירה את
המחיר הסופי.
(מענה למטרה 2
ותורמת למטרה 5)

דוגמה לפלט:

	FinalPrice
1	110.00

פונקציה המחזירה טבלה -

תיאור: פונקציה המחזירה את כל המוצרים (הפעילים) ללא אלרגן מסוים. (מענה למטרה 3)

```
-- פונקציה שמחזירה את כל המוצרים ללא אלרגיה מסוימת--
CREATE FUNCTION dbo.GetProdWithoutAllergy(@AllergyName nvarchar(25))
RETURNS TABLE
AS
RETURN(
select p.ProductId, p.ProductName, p.Price, c.CategoryName, p.Units, a.AllergyName
from Products p join Categories c on p.CategoryId = c.CategoryId left join ProductsAllergies pal on p.ProductId = pal.ProductId
left join Allergies a on a.AllergyId = pal.AllergyId
where p.Status = 1 AND p.ProductId not in (select p.ProductId
from Products p join ProductsAllergies pal on p.ProductId = pal.ProductId
join Allergies a on a.AllergyId = pal.AllergyId
where a.AllergyId = (select TOP 1 a.AllergyId
from Allergies a
where a.AllergyName = @AllergyName))
)
```

```
-- קריאה לפונקציה--
select *
from dbo.GetProdWithoutAllergy('גלוטן')
```

	ProductId	ProductName	Price	CategoryName	Units	AllergyName
1	1	מוס שוקולד בלגי אישי	39.9	קינוחים אישיים	50	NULL
2	4	פאי פיצוחים	25	קינוחים אישיים	100	בוטנים
3	5	טארט לימון אישי	22	קינוחים אישיים	50	NULL
4	8	קרם ברולה	25	קינוחים אישיים	60	NULL
5	9	טארט פירות אישי	24.9	מוסים	10	NULL
6	10	דניש גבינה	24.9	עוגות גבינה	30	חלב
7	12	דניש פירות	24.9	מוסים	30	NULL
8	14	קראנץ' קינמון	22	מאפים מתוקים	12	NULL
9	17	סופגניה במילוי ריבת...	8	חגים	80	חלב
10	18	טריו מוס שוקולד	30	קינוחים אישיים	45	NULL
11	19	טריקולד	39.9	מוסים	32	NULL

דוגמה לפלט:

פונקציה המחזירה טבלה -

תיאור: פונקציה המחזירה את כל המוצרים (הפעילים) ללא האלרגנים שהגדיר שהוא מעוניין להימנע. (מענה למטרה 2 ו-3)

-- פונקציה שמקבלת קוד לקוח ומחזירה את המוצרים ללא האלרגנים שהוא מעוניין להימנע--

```
CREATE FUNCTION dbo.GetPriorityProdCus(@CustomerId INT)
```

```
RETURNS TABLE
```

```
AS
```

```
RETURN(
```

```
select p.ProductId, p.ProductName, p.price, c.CategoryName, a.AllergyName
```

```
from Products p left join ProductsAllergies pal on p.ProductId = pal.ProductId
```

```
left join Allergies a on a.AllergyId = pal.AllergyId
```

```
left join Categories c on c.CategoryId = p.CategoryId
```

```
where @CustomerId IS NOT NULL AND EXISTS(select 1 from Customers where CustomerId = @CustomerId)And
```

```
p.Status =1 AND
```

```
p.ProductId not in(select p.ProductId
```

```
from Customers c join CustomerAllergyPriority cap on c.CustomerId = cap.CustomerId
```

```
join Allergies a on a.AllergyId = cap.AllergyId
```

```
join ProductsAllergies pal on pal.AllergyId = a.AllergyId
```

```
join Products p on p.ProductId = pal.ProductId
```

```
where c.CustomerId = @CustomerId)
```

```
)
```

דוגמה לפלט:

-- קריאה לפונקציה--

```
select * from dbo.GetPriorityProdCus(2)
```

	ProductId	ProductName	price	CategoryName	AllergyName
1	6	עוגת קפה	33	מאפים מתוקים	NULL
2	24	טראפל שוקולד	5	קינורחים אישיים	אגוזים
3	25	רוגלאך וניל	15.9	מאפים מתוקים	NULL
4	27	עוגת נרגס	39.9	מאפים מתוקים	NULL

פונקציה המחזירה טבלה -

תיאור: פונקציה המחזירה את כל אלרגנים שיש במוצר. (מענה למטרה 3 ו-1)

```
-- פונקציה המחזירה טבלה ובה כל האלרגנים שהמוצר מכיל--  
CREATE FUNCTION dbo.GetProductAllergies(@ProductName NVARCHAR(25))  
RETURNS TABLE  
AS  
    RETURN(select p.ProductId, p.ProductName, a.AllergyName  
        from Products p join ProductsAllergies pal on p.ProductId = pal.ProductId  
        join Allergies a on a.AllergyId = pal.AllergyId  
        where pal.ProductId = (select p.ProductId  
                                from Products p  
                                where p.ProductName = @ProductName))
```

דוגמה לפלט:

	ProductId	ProductName	AllergyName
1	3	עוגת גבינה	גלוטן
2	3	עוגת גבינה	חלב
3	3	עוגת גבינה	בוטנים

```
-- קריאה לפונקציה--  
select*  
from dbo.GetProductAllergies('עוגת גבינה')
```

פונקציה המחזירה טבלה -

תיאור: פונקציה המחזירה את כל המוצרים (הפעילים) השייכים לקטגוריה מסוימת. (מענה למטרה 1)

שליפת כל המוצרים המשתייכים לקטגוריה המתקבלת--

```
CREATE FUNCTION dbo.GetCategoryProducts(@CategoryName nvarchar(25))  
RETURNS TABLE  
AS  
RETURN(select p.ProductId, ProductName, p.Price, c.CategoryName, p.Units, p.Status  
        from Products p join Categories c on p.CategoryId = c.CategoryId  
        where p.Status = 1 AND p.CategoryId = (select c.CategoryId from Categories c  
        where c.CategoryName = @CategoryName))
```

קריאה לפונקציה--

```
select *  
from dbo.GetCategoryProducts('קינוחים אישיים')
```

דוגמה לפלט:

	ProductId	ProductName	Price	CategoryName	Units	Status
1	1	מוס שוקולד בלגי אישי	12.9	קינוחים אישיים	48	1
2	4	פאי פיצוחים	25	קינוחים אישיים	100	1
3	5	טארט לימון אישי	22	קינוחים אישיים	50	1
4	8	קרם ברולה	25	קינוחים אישיים	60	1
5	18	טריו מוס שוקולד	30	קינוחים אישיים	45	0
6	22	טראפל שוקולד	5	קינוחים אישיים	35	1

פונקציה המחזירה טבלה -

תיאור: פונקציה המחזירה את כל המוצרים (הפעילים) שעולים החל ממחיר מסוים. (מענה למטרה 1)

--שליפת כל המוצרים שעולים החל ממחיר מסוים--

```
CREATE FUNCTION dbo.GetProductsAbovePrice(@Price MONEY)
```

```
RETURNS TABLE
```

```
AS
```

```
RETURN(select p.ProductId, p.ProductName, p.Price, c.CategoryName, p.Units, p.Status  
from Products p join Categories c on p.CategoryId = c.CategoryId  
where p.Status = 1 AND p.Price >= @Price)
```

דוגמה לפלט:

--קריאה לפונקציה--

```
select *
```

```
from dbo.GetProductsAbovePrice(24.9)
```

	ProductId	ProductName	Price	CategoryName	Units	Status
1	3	עוגת גבינה	29.9	מוסים	30	1
2	4	פאי פיצוחים	25	קינוחים אישיים	100	1
3	8	קדם ברולה	25	קינוחים אישיים	60	1
4	9	טארט פירות אישי	24.9	מוסים	10	1
5	10	דניש גבינה	24.9	עוגות גבינה	30	1
6	12	דניש פירות	24.9	מוסים	30	1
7	18	טריו מוס שוקולד	30	קינוחים אישיים	45	0
8	19	טריקולד	39.9	מוסים	32	1

פונקציה המחזירה טבלה -

תיאור: פונקציה המחזירה עבור כל קטגוריה את המוצר היקר ביותר שלה (מוצר פעיל).
(מענה למטרה 1)

```
CREATE FUNCTION dbo.GetExpensiveCategoryProd()  
RETURNS TABLE  
AS  
RETURN(select p.ProductId, p.ProductName, p.Price, c.CategoryName, p.Units, p.Status  
from Products p join Categories c on p.CategoryId = c.CategoryId  
where p.Status = 1 AND p.Price >=ALL (select p2.Price  
from Products p2  
where p2.Status = 1 AND p2.CategoryId = p.CategoryId))
```

-- קריאה לפונקציה--

```
select *  
from dbo.GetExpensiveCategoryProd()
```

דוגמה לפלט:

	ProductId	ProductName	Price	CategoryName	Units	Status
1	4	פאי פיצוחים	25	קינוחים אישיים	100	1
2	8	קדם ברולה	25	קינוחים אישיים	60	1
3	10	דניש גבינה	24.9	עוגות גבינה	30	1
4	17	סופגניה במילוי ריבת חלב	8	חגים	80	1
5	19	טריקולד	39.9	מוסים	32	1
6	23	רוגלאך וניל	15.9	מאפים מתוקים	30	1

```

CREATE PROCEDURE AddProduct
@ProductName nvarchar(25),
@Price float,
@CategoryId INT,
@Units INT
AS
BEGIN
    BEGIN TRANSACTION
    BEGIN TRY
        IF(@ProductName IS NULL OR @ProductName = '')
        BEGIN
            PRINT 'שם מוצר אינו חוקי'
            ROLLBACK TRANSACTION
            RETURN
        END
        IF(@Price IS NULL OR @Price <=0)
        BEGIN
            PRINT 'מחיר אינו חוקי'
            ROLLBACK TRANSACTION
            RETURN
        END
        IF NOT EXISTS(select 1 from Categories where CategoryId = @CategoryId)
        BEGIN
            PRINT 'קוד קטגוריה לא קיים'
            ROLLBACK TRANSACTION
            RETURN
        END
        IF(@Units IS NULL OR @Units <=0)
        BEGIN
            PRINT 'כמות אינה חוקית'
            ROLLBACK TRANSACTION
            RETURN
        END
        INSERT INTO Products (ProductName, Price, CategoryId, Units)VALUES(@ProductName, @Price, @CategoryId, @Units)
        PRINT 'נוסף בהצלחה!'
        COMMIT TRANSACTION
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION
        PRINT 'ארעה שגיאה, המוצר לא נוסף'
    END CATCH
END

```

פרוצדורה עם טרנזקציה-

תיאור: הוספת מוצר. (מענה למטרה 1)

```
exec AddProduct 'טראפל שוקולד',5,2, 35
```

פרוצדורות -

תיאור: הוספת אלרגנים למוצרים. (מענה למטרה 1 ו-3)

```
CREATE PROCEDURE AddAlergiesToProduct
@ProductId INT,
@AllergyId INT
AS
BEGIN
    IF NOT EXISTS(select 1 from Products where ProductId = @ProductId)
    BEGIN
        PRINT 'קוד מוצר לא קיים'
        RETURN
    END
    IF NOT EXISTS(select 1 from Allergies where AllergyId = @AllergyId)
    BEGIN
        PRINT 'קוד אלרגיה לא קיים'
        RETURN
    END
    IF EXISTS(select 1 from ProductsAllergies where ProductId = @ProductId and AllergyId = @AllergyId)
    BEGIN
        PRINT 'אלרגן כבר קיים למוצר זה'
        RETURN
    END
    INSERT INTO ProductsAllergies(ProductId, AllergyId)VALUES(@ProductId, @AllergyId)
    PRINT 'נוסף בהצלחה!'
```

```
exec AddAlergiesToProduct 21,1
```

```
exec AddAlergiesToProduct 18,2
```

פרוצדורות -

תיאור: הוספת הזמנה. (מענה למטרה 2)

```
CREATE PROCEDURE AddOrder
@CustomerId INT,
@CustomerComment NVARCHAR(MAX) = NULL
AS
BEGIN
    IF NOT EXISTS(SELECT 1 FROM Customers where CustomerId = @CustomerId)
        BEGIN
            PRINT 'אינך רשום במערכת, אנא צור חשבון'
        END
        INSERT INTO Orders (CustomerId, CustomerComments) VALUES(@CustomerId, @CustomerComment)
        PRINT 'נוסף בהצלחה!'
    END
```

-- הוספת הזמנות

```
exec AddOrder 3, "אני אלרגית לבוטנים, אנא ודאו שאין עקבות של אגוזים"
exec AddOrder 3
```

פרוצדורות -

תיאור: הוספת פריט להזמנה. (מענה למטרה 2)

```
CREATE PROCEDURE AddOrderItem
@OrderId INT,
@ProductId INT,
@Quantity INT
AS
BEGIN
    DECLARE @UnitPriceAtPurchase Float;
    IF NOT EXISTS(select 1 from Orders where OrderId = @OrderId)
    BEGIN
        PRINT 'קוד הזמנה לא קיים'
        RETURN
    END
    IF NOT EXISTS(select 1 from Products where ProductId = @ProductId)
    BEGIN
        PRINT 'קוד מוצר לא קיים'
        RETURN
    END
    IF (@Quantity <=0)
    BEGIN
        PRINT 'כמות לא חוקית'
        RETURN
    END
    SELECT @UnitPriceAtPurchase = Price
    FROM Products
    WHERE ProductId = @ProductId

    INSERT INTO OrderDetails(OrderId, ProductId, Quantity, UnitPriceAtPurchase)VALUES(@OrderId, @ProductId, @Quantity, @UnitPriceAtPurchase)
    PRINT 'נוסף בהצלחה!'
END
```

-- הוספת מוצרים להזמנה

exec AddOrderItem 8, 1, 3

exec AddOrderItem 8, 4, 1

exec AddOrderItem 9, 17, 1

פרוצדורות -

```
CREATE PROCEDURE AddCustomerAllergiesPriority
@CustomerId INT,
@AllergyId INT
AS
BEGIN
    IF NOT EXISTS(select 1 from Customers where CustomerId = @CustomerId)
    BEGIN
        PRINT 'לקוח לא קיים במערכת, אנא צור חשבון'
        RETURN
    END
    IF NOT EXISTS(select 1 from Allergies where AllergyId = @AllergyId)
    BEGIN
        PRINT 'קוד אלרגן לא קיים'
        RETURN
    END
    IF EXISTS(select 1 from CustomerAllergyPriority where CustomerId = @CustomerId and AllergyId = @AllergyId)
    BEGIN
        PRINT 'אלרגן כבר רשום ללקוח זה'
        RETURN
    END
    INSERT INTO CustomerAllergyPriority(CustomerId, AllergyId)VALUES(@CustomerId, @AllergyId)
    PRINT 'נוסף בהצלחה!'
END
```

תיאור: הוספת אלרגנים מהם הלקוח מבקש להימנע (עקב רגישות או אלרגיה).
(מענה למטרה 3 ו-2)

```
exec AddAlergiesToProduct 3,4
```

פרוצדורות -

תיאור: הוספת לקוח. (מענה למטרה 2)

-- הוספת לקוח--

```
CREATE PROCEDURE AddCustomer
@FirstName nvarchar(25),
@LastName nvarchar(25),
@ClubMember bit = 1
AS
BEGIN
    IF(@FirstName = '' OR @LastName = '' OR @FirstName IS NULL OR @LastName IS NULL)
        BEGIN
            PRINT 'השם שהוזן לא חוקי'
            RETURN
        END
    INSERT INTO Customers(FirstName, LastName, ClubMember) VALUES(@FirstName, @LastName, @ClubMember)
    PRINT 'נוסף בהצלחה!'
END
```

```
exec AddCustomer 'שלומי', 'ביטון'
```

פרוצדורות -

תיאור: הוספת אלרגן. (מענה למטרה 3)

```
CREATE PROCEDURE AddAllergy
@AllergyName nvarchar(25)
AS
BEGIN
    IF (@AllergyName IS NULL OR LTRIM(RTRIM(@AllergyName)) = '')
    BEGIN
        PRINT 'ערך לא חוקי'
    END
    IF EXISTS(select 1 from Allergies where AllergyName = @AllergyName)
    BEGIN
        PRINT 'האלרגן כבר מעודכן במערכת'
        RETURN
    END
    INSERT INTO Allergies(AllergyName)VALUES(@AllergyName)
    PRINT 'נוסף בהצלחה!'
END
```

```
exec AddAllergy 'בוטנים'
```

פרוצדורות -

תיאור: עדכון מחיר מוצר. (מענה למטרה 1)

```
CREATE PROCEDURE UpdatePrice
@ProductId INT,
@NewPrice FLOAT
AS
BEGIN
    IF(@NewPrice <=0 or @NewPrice IS NULL)
    BEGIN
        PRINT 'מחיר לא חוקי'
    END
    IF NOT EXISTS(select 1 from Products where ProductId = @ProductId)
    BEGIN
        PRINT 'קוד מוצר לא קיים'
        RETURN
    END
    UPDATE Products set Price = @NewPrice where ProductId = @ProductId
    PRINT 'המחיר שונה בהצלחה!'
END
```

```
exec UpdatePrice 1,12.9
```

פרוצדורות -

תיאור: מחיקת לקוח. (מענה למטרה 2)

```
-- מחיקת לקוח
CREATE PROCEDURE DeleteCustomer
@CustomerId INT
AS
BEGIN
    If(@CustomerId IS NULL OR @CustomerId = '' )
    BEGIN
        PRINT 'ערך לא חוקי'
        RETURN
    END
    IF NOT EXISTS(select 1 from Customers where CustomerId = @CustomerId)
    BEGIN
        PRINT 'קוד לקוח לא קיים'
        RETURN
    END
END
```

```
-- מחיקת מוצר (שינוי סטטוס) והפעלת הטריגר--
exec DeleteCustomer 10
```

פרוצדורות -

תיאור: מחיקת מוצר. (מענה למטרה 1)

```
CREATE PROCEDURE DeleteProduct
@ProductId INT
AS
BEGIN
    IF NOT EXISTS(select 1 from Products where ProductId = @ProductId)
        BEGIN
            PRINT 'קוד מוצר לא קיים'
            RETURN
        END
    delete from Products
    where ProductId = @ProductId
END
```

```
exec DeleteProduct 25
```

טריגרים -

תיאור: טריגר למניעת הזמנת מוצר אם אין מספיק במלאי. (מענה למטרה 2)

```
CREATE TRIGGER PreventOverOrder
on OrderDetails
INSTEAD OF INSERT
AS
BEGIN
    IF EXISTS(select 1 from inserted i join Products p on i.ProductId = p.ProductId where p.Units > i.Quantity)
    BEGIN
        PRINT 'אין מספיק במלאי'
        ROLLBACK
        RETURN
    END
    INSERT INTO OrderDetails(ProductId, Quantity, UnitPriceAtPurchase)
    select ProductId, Quantity, UnitPriceAtPurchase FROM inserted
END
```

הפעלת הטריגר:

```
exec AddOrderItem 9,1,100
```

טריגרים -

תיאור: טריגר לעדכון כמות המלאי של המוצר לאחר ההזמנה. (מענה למטרה 1)

```
CREATE TRIGGER UpdateUnits
on OrderDetails
AFTER INSERT
AS
BEGIN
    UPDATE p
    SET P.Units = P.Units - OD.Quantity
    from Products p
    INNER JOIN inserted OD on OD.ProductId = p.ProductId
END
```

הפעלת הטריגר:

```
exec AddOrderItem 9, 1, 2
```

טריגרים -

תיאור: טריגר לעדכון סטטוס הלקוח בעת מחיקה. (מענה למטרה 2)

```
-- הפעלת הפרוצדורה
exec AddCustomer 'ביטון', 'שלומי'
-- מחיקת לקוח
CREATE PROCEDURE DeleteCustomer
@CustomerId INT
AS
BEGIN
    If(@CustomerId IS NULL OR @CustomerId = '' )
    BEGIN
        PRINT 'ערך לא חוקי'
        RETURN
    END
    IF NOT EXISTS(select 1 from Customers where CustomerId = @CustomerId)
    BEGIN
        PRINT 'קוד לקוח לא קיים'
        RETURN
    END
END
```

```
-- מחיקת מוצר (שינוי סטטוס) והפעלת הטריגר
exec DeleteCustomer 10
```

הפעלת הטריגר:

טריגרים -

תיאור: טריגר לעדכון סטטוס מוצר בעת מחיקת מוצר. (מענה למטרה 1)

```
-- טריגר שינוי סטטוס הזמנה--
CREATE TRIGGER ChangeProductStatus
on Products
INSTEAD OF DELETE
AS
BEGIN
    UPDATE p
    set p.Status = 0
    from Products p join deleted d on p.ProductId = d.ProductId
END
```

הפעלת הטריגר:

```
exec DeleteProduct 25
```